Stale but on Time: What Asynchronous Memory Recall Costs a Real-Time Conversational Agent

Anton Chernov *anton@aenimalabs.com*
Aenima Labs

Abstract

A voice agent has to reply quickly, and retrieving long-term memories on the reply path takes time. A common remedy moves recall off that path: background jobs prepare context, and the reply uses whatever is ready. The price is staleness, because that context was computed before the listener's latest words. We measure the price. On 4,086 replayed utterances of LoCoMo, recall computed one utterance early shares 0.50 of its top-5 memories with fresh retrieval; computed on half of the incoming utterance, it shares 0.77. In end-to-end trials with a small local model, a never-wait agent passes 19% of the cases that need a memory, against 75% for blocking retrieval and 82% for a search tool. No trial passed without the right memory in context, and the loss was the same with the model's reasoning mode on, with a second local model and with a hosted model. What followed a missing memory did differ: one model sometimes invented an answer, and only the stronger settings searched past stale notes, at the price of a second model call. We then built the design into the product it was specified for. There, the never-wait record passed 29% and retrieval that blocked for a median of 97 ms passed 84%. Across our set-ups the wait for retrieval was between 8 and 336 ms, and at no moment had the never-wait agent answered more than 4 points more of the trials correctly. Asynchronous recall should carry only work that is too slow to block on, start on partial transcripts, and expire within one exchange.

1 Introduction

People leave gaps of roughly 200 ms between turns in conversation (Stivers et al., 2009; Levinson & Torreira, 2015). Spoken dialogue systems are slower, and their delay is one of the first things users notice (Skantze, 2021). In a pipeline of speech recognition, language model and speech synthesis, every step on the path from the end of the listener's utterance to the first audio of the reply adds to that delay.

Long-term memory puts one more step on the path. An agent that knows a person's history has to bring the right memories into the model's context before it answers. Xiao et al. (2026) argue that this is not a search problem: search starts from a query, and knowing what to ask for already requires memory. They call for *proactive recall*, where memories are retrieved from the ambience of the moment instead of from a question.

There are three ways to place recall in a turn (Figure 1). *Blocking retrieval* runs recall when the listener stops and waits for it. A *search tool* lets the model decide to search, which costs one extra model call before recall begins. *Asynchronous* recall runs in the background and the reply reads whatever result is ready, without waiting.

The asynchronous design is attractive, and several recent systems use a version of it (Christakopoulou et al., 2024; Qiu et al., 2026; Gan et al., 2025; Chien et al., 2026; Arora et al., 2025). They report lower latency, cache hit rates, or answer quality close to a slower baseline. What they leave open is the cost that the design creates by construction. A result that is ready when the listener stops was computed from an earlier state

Use of AI. This paper was produced with an AI assistant (Claude, Anthropic). Under the author's direction the assistant wrote the experiment code and the product changes tested in Experiment 3, authored the fictional test data, ran the experiments, and drafted the text. The author takes responsibility for the content.

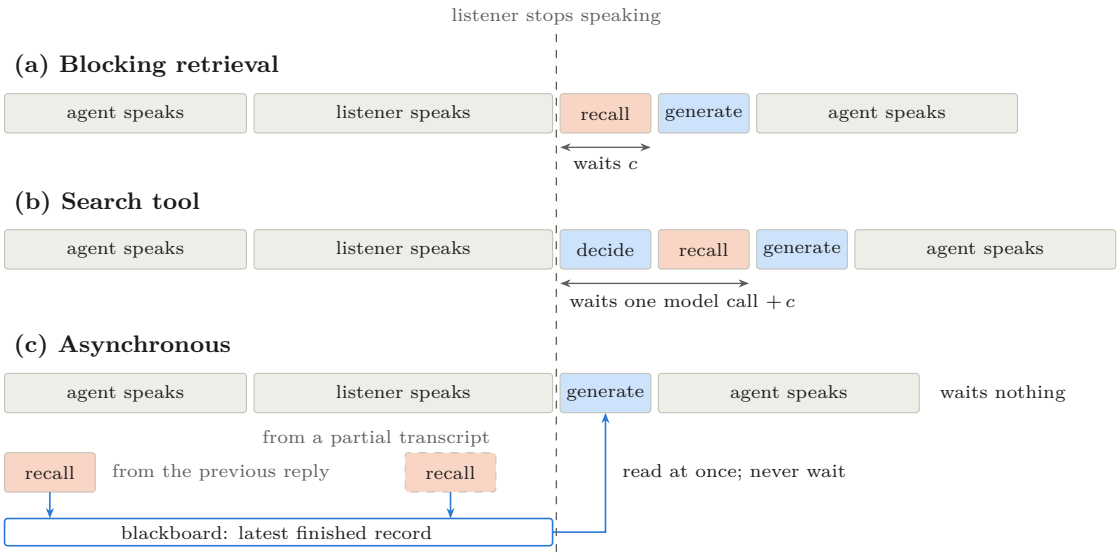


Figure 1: Three places to put recall in a voice turn. In (a) and (b) the reply waits for recall, which takes c seconds. In (c) it never waits: recall jobs run while the agent and the listener speak, and the reply reads the latest finished result. That result was computed before the listener finished, so it can be stale.

of the conversation. How different is it from the result that fresh retrieval would return, and how often does that difference change what the agent says?

We ask this question for a voice companion that we are building, Kodama, which speaks about one person's life from a store of that person's memories. Its design moves all slow recall into background jobs. In our deployment the median voice turn took 3.3 s over 22 measured turns, and a model call that included a memory search could alone take more than four seconds, so the motivation is real. We measured what the background jobs would cost on a reference implementation, then built them into the product and measured again. After a precise statement of the design (Section 3), this paper reports three findings.

1. *Early recall is half different.* On 4,086 utterances of human conversation, recall computed one utterance early shares 0.50 of its top five memories with fresh retrieval, and three utterances early 0.18, close to the 0.08 of unrelated context. Starting on half of the incoming utterance brings it back to 0.77 (Section 5).

2. *The difference decides the reply.* Over 1,200 sealed trials, a never-wait agent passes 19% of the cases that need a memory, blocking retrieval 75% and a search tool 82%. No trial passed without the right memory in context. With reasoning on, with a second local model or with a hosted model, the loss barely moves; whether the agent then abstains or invents, and whether it searches past stale notes, changes with the model (Section 6).

3. *The product behaves the same way.* In 1,000 trials against Kodama itself, the never-wait record passes 29% and blocking retrieval 84%, at a median cost of 97 ms on the reply path. The search tool that the product shipped with passes 63% and makes the reply about two seconds slower (Section 7).

4. *Waiting is cheap, and not waiting is never usefully faster.* The wait for retrieval was 15 ms with a local embedding model and 336 ms with a hosted one. Counted as correct answers by time, the never-wait agent is at most 4 points ahead, for 0.2 s (Section 8).

These results are less favourable to asynchronous recall than we expected. Section 9 draws out what they imply for design.

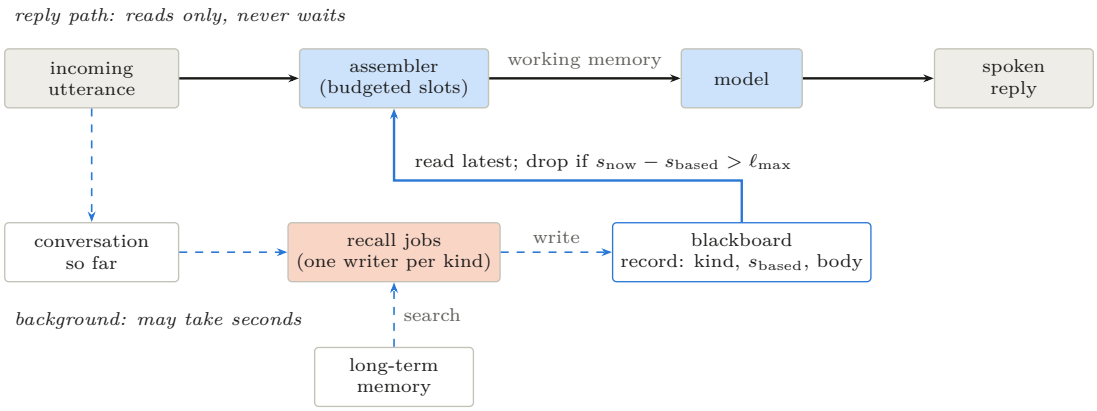


Figure 2: Asynchronous context assembly. The reply path (top) only reads. Recall jobs (bottom) are started by conversation events and write records to a blackboard. Each record carries the utterance number s_{based} it was computed from, and the assembler drops a record that is more than $\ell_{\max}$ utterances old.

2 Related work

Memory for conversational agents. Retrieval-augmented generation retrieves with the user's query before generating (Lewis et al., 2020), and agent memory systems add stores that the agent writes and searches itself (Packer et al., 2023; Park et al., 2023; Chhikara et al., 2025; Xu et al., 2025; Hu et al., 2025). Benchmarks such as LoCoMo (Maharana et al., 2024) and LongMemEval (Wu et al., 2024) score answers to questions asked after the conversation, and do not model when, within a live turn, a memory has to be available. Xiao et al. (2026) argue that such benchmarks measure search and that memory should be recalled proactively from context; Liu et al. (2025) study agents that form thoughts in parallel with a conversation and decide when to voice them.

Taking slow work off the reply path. The closest systems, compared in Table 4 of the appendix, are Talker–Reasoner (Christakopoulou et al., 2024), VoiceAgentRAG (Qiu et al., 2026), PMFR (Gan et al., 2025), Stream RAG (Arora et al., 2025), MoshiRAG (Chien et al., 2026; Défossez et al., 2024) and sleep-time compute (Lin et al., 2025). In each, a slow process works beside a fast one and hands over its result through shared state, an idea that goes back to the blackboard of Hearsay-II (Erman et al., 1980); Ginart et al. (2024) give an event-driven architecture for it. The closest is VoiceAgentRAG, whose background agent predicts follow-up topics and prefetches documents into a cache, with blocking retrieval as the fallback on a miss. Our design belongs to this family and does not claim the idea of asynchronous retrieval. The difference is in the question. These papers show that latency falls; we measure what is lost when the reply never waits, as a function of how far behind the background result is.

3 Asynchronous context assembly

Setting. A conversation is a sequence of utterances $u_1, u_2, \ldots$ that alternate between listener and agent. The agent has a memory store M. The *ambience* a_t^w is the text of the last w utterances up to and including u_t. A recall function $R(M, a)$ returns the k memories that best match an ambience; it may be a fast index lookup or a slow chain of model calls. We write c for the time one recall takes.

Three designs. Suppose the listener has just finished u_t. *Blocking retrieval* computes $R(M, a_t^w)$ and then generates; the reply is delayed by c. A *search tool* first asks the model whether to search and for what, then runs recall on the model's query; the reply is delayed by one model call plus c. *Asynchronous assembly* computes nothing on the reply path. Recall jobs run in the background and the reply uses a result that already exists.

Blackboard and staleness bound. Figure 2 shows the parts. Each kind of background work, such as recall of events or recall of facts about people, has one writer. When a job that started from ambience a_s^w finishes, it replaces the record of its kind on the blackboard with $(\text{kind}, s, \text{body})$. At reply time t the assembler reads each record once. It uses the body if $t - s \leq \ell_{\max}$ and otherwise leaves that slot of the working memory empty. We call $L = t - s$ the *lag* of the record, counted in utterances. The assembler fills the remaining slots, such as instructions and a session summary, from sources that need no model call, under a fixed budget.

Two properties follow. The reply path costs one read, whatever c is. And c does not disappear: it reappears as lag, because a slower job has to start earlier to be finished in time. A failed or late job produces an empty or old slot, never a delayed reply.

When jobs start. A job can start whenever the conversation changes. The agent's own reply exists as text before it is spoken, so a job that starts then has the whole of the agent's speech and the listener's next utterance to finish, and yields lag 1. If the speech recogniser streams partial transcripts, a job can also start in the middle of the listener's utterance. Its ambience then holds the first part of u_t, which we describe by the fraction p of the words it has seen.

Implementation. The design was specified for Kodama. Experiments 1 and 2 use a small reference implementation of the rules above, so that each condition can be isolated. For Experiment 3 we built the rules into the product: the blackboard, a recall job that starts after every saved utterance, an assembler slot that reads its record under the staleness bound, and, for comparison, the same recall run on the reply path. Jobs that start on partial transcripts are not built, because the product's speech recogniser returns whole utterances.

4 Evaluation method

The experiments compare conditions that differ only in *when* recall sees the conversation, so everything else must be held fixed and the model's context must be recorded on each trial. We use Circumstances, an open-source framework for testing complete agent systems in scripted situations. A *pack* holds rules, cases with scripted actions, and evaluators with known-pass and known-fail self-tests. A *subject plugin* connects the agent under test and returns each trial as a list of typed events: messages, tool calls, snapshots of the assembled context, observations and metrics. A single run says little about a stochastic agent (Yao et al., 2024), so the runner repeats every case, grades the stored events and seals the run in a bundle of hashed files, which can be regraded without running the agent again. Each memory condition is a different subject on the same pack. The context snapshot records which memories the model saw, which lets us test whether success depends on anything else. The framework separates text matches, its weakest tier of evidence, from checks on recorded state, and we say which tier each result rests on.

5 Experiment 1: how stale is recall that was computed early?

5.1 Setup

Data. LoCoMo contains ten long conversations between two people, each spread over many sessions (Maharana et al., 2024). We replay them utterance by utterance. For an utterance in session j, the memory store holds every utterance of sessions 1 to $j - 1$ of the same conversation, one memory per utterance. We evaluate every utterance from the seventh of a session onward, in sessions two and later, so that all lags up to six exist. This gives 4,086 utterances in 262 sessions; the store holds between 16 and 665 memories (median 296).

Recall. We fuse BM25 (Robertson & Zaragoza, 2009) and cosine similarity of `nomic-embed-text` embeddings (Nussbaum et al., 2024) by reciprocal rank fusion with constant 60 (Cormack et al., 2009), and keep the top $k = 5$. The same function serves every condition.

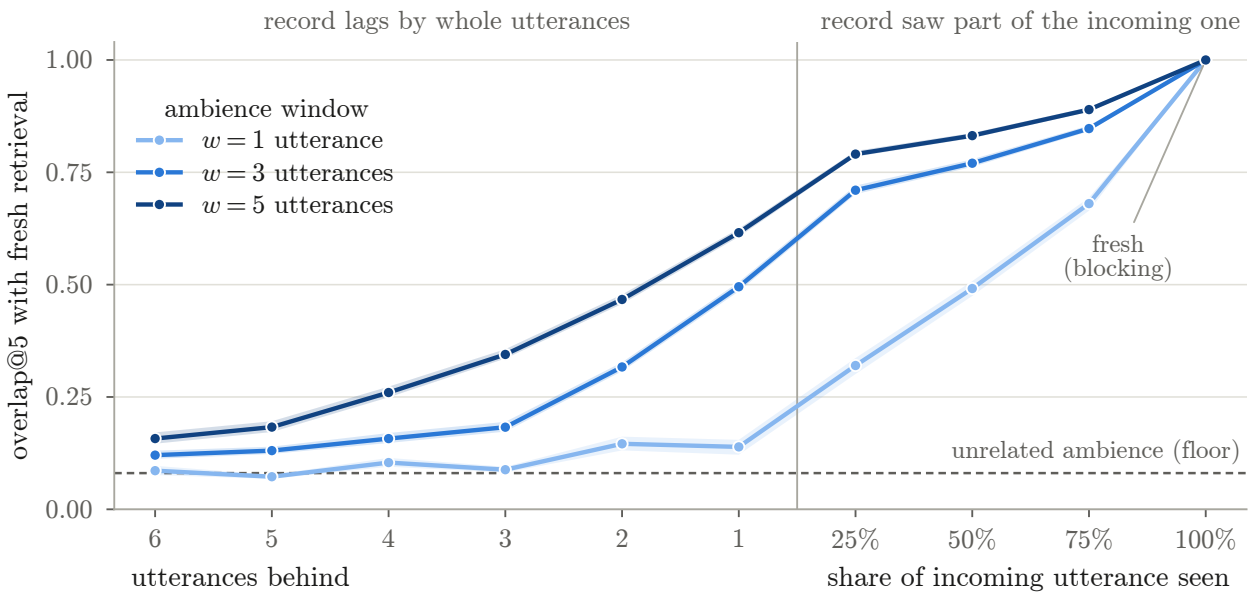


Figure 3: Agreement between early recall and fresh recall on LoCoMo (4,086 utterances). Left of the divider the record is whole utterances behind; right of it, the record has seen part of the incoming utterance. Shaded bands are 95% intervals and are narrower than the markers in most places.

Conditions. *Fresh* recall uses the ambience a_t^w, which includes the incoming utterance; it is what blocking retrieval would return. A *lag-L* record uses a_{t-L}^w. A *prefix-p* record uses the $w-1$ utterances before u_t and the first p per cent of the words of u_t. As a floor, *unrelated ambience* uses the fresh ambience of a random utterance from another session of the same conversation. We run $w \in \{1, 3, 5\}$ and report $w = 3$ unless stated.

Measure. For a condition with top-k set T and fresh set T^*, the overlap is $|T \cap T^*|/k$. It needs no relevance labels: it measures how far a design that never waits departs from one that does. It does not say that the fresh set is correct, a limit we return to in Section 10. Intervals are 95% percentile intervals from a bootstrap (Efron, 1979) that resamples the ten conversations.

5.2 Results

Figure 3 shows the result, and Table 5 in the appendix gives the numbers.

One utterance of lag replaces half of the context. At lag 1 the overlap is 0.50 [0.49, 0.51]. It falls to 0.32 at lag 2 and 0.18 at lag 3, against a floor of 0.08 [0.07, 0.09] for unrelated ambience. The memory that fresh recall ranks first is still among the lag-1 top five for 69% of utterances.

A partial transcript recovers much of it. A record that has seen the first quarter of the incoming utterance reaches 0.71; half of it gives 0.77 and three quarters 0.85. The fresh top memory is in the half-utterance top five 92% of the time.

The loss is not confined to topic shifts, and part of it may not matter. Lag-1 overlap is 0.53 in the quarter of utterances that continue the preceding ones most closely and 0.48 in the most abrupt quarter: almost every new utterance moves the retrieved set. Measured against retrieval with the utterance that actually came next, lag 1 keeps 79% of the margin that fresh recall has over the floor, so part of what lag replaces was not going to be needed (Appendix B).

A slow job has to start early, and then it is stale. Appendix B.1 turns the cost c of one recall job into the overlap a never-wait agent can expect, with a simple model of speaking time (Figure 7). If jobs start only at the ends of utterances, expected overlap stays near the lag-1 value of 0.50 for any cost up to about five

seconds and then falls towards the floor. If jobs also start on partial transcripts, they do better only while they finish within the rest of the utterance (0.82 at 1.3 s, 0.59 at 5.4 s). However cheap recall is, a never-wait agent stops near 0.85 with partial transcripts and near 0.50 without.

6 Experiment 2: does staleness change what the agent says?

6.1 Setup

Memory and cases. We wrote a fictional store of 48 short notes about one invented person, a bookbinder born in 1941, in the form Kodama uses: a few sentences each about her childhood, work, family, habits and possessions. The code and the result files call this store a *memorome*, the term of Xiao et al. (2026) for the whole collection of one person's memories. All names, places and dates are invented so that no answer can be guessed. We then wrote 40 cases. Each opens with two scripted utterances, one from the listener and one from the agent, and ends with the listener's utterance that the agent must answer. There are four classes of ten:

- *Asked, topic continues.* The listener asks for a remembered fact on the topic of the lead-in.
- *Asked, topic shifts.* The listener asks for a fact unrelated to the lead-in.
- *Unprompted.* The listener asks nothing but says something to which one memory is clearly relevant. Half of these lead-ins are on the topic and half are not.
- *Must abstain.* The listener asks for a fact the store does not contain, in most cases with a nearby note that invites a guess.

Grading. In the first three classes a trial passes if the reply contains one of a small set of strings that identify the remembered fact, for example a name or a year that appears in the target note and nowhere in the lead-in. The framework classes this as a text match. In the abstain class the agent returns its reply together with a flag that says whether it declined to answer, and a trial passes if the flag is set; this is a check on recorded state. Section 6.2 reports an audit of both, and Section 6.3 two corrections that a later run made necessary.

Conditions. One agent is run six ways. The prompt, model, retriever, window ($w = 3$) and number of memories shown ($k = 4$) are the same throughout.

- *No memory.* Nothing is loaded.
- *Search tool.* Nothing is loaded; the model may call a search tool once before it replies.
- *Blocking retrieval.* Recall on the ambience including the incoming utterance, on the reply path.
- *Async, lag 1.* The record was computed from the lead-in only.
- *Async, prefetch 50%.* The record was computed from the lead-in and the first half of the incoming utterance.
- *Async lag 1 + search.* The lag-1 record is loaded and the search tool is also offered.

Model and runs. The agent is `gemma4:e4b` (8 billion parameters, 4-bit quantised) served by Ollama 0.40 on one laptop, at temperature 0.7 with its reasoning mode switched off. Each case runs five times with fixed seeds: 200 trials per condition and 1,200 in all. We debugged the harness on four cases before the full run, which led us to switch off the reasoning mode and to use the model's native tool calls. Cases and evaluators were not changed afterwards. This is the main run; Section 6.3 reports three further full runs of the same pack, and no other full run was made. Intervals are 95% percentile intervals from a bootstrap over cases.

Latency. We report the delay each condition adds before the final generation call, which is the same call in every condition. Time to a complete reply also depends on the length of the reply (Table 8).

Table 1: End-to-end results: percentage of trials passed (200 trials per condition, five per case). "Recall" pools the three classes in which a memory should be used, with a 95% interval. "Memory in context" is the share of those trials in which a target note was among the memories shown to the model. "Added delay" is the time before the final generation call starts.

Condition	Recall (30 cases)	Asked, continues	Asked, shifts	Un-prompted	Must abstain	Memory in context	Added delay (ms) median	90th pct.
No memory	0 [0, 0]	0	0	0	100	–	0	0
Search tool	82 [71, 91]	80	92	74	84	85	426	820
Blocking retrieval	75 [59, 89]	62	68	94	90	83	15	17
Async, prefetch 50%	43 [27, 60]	34	34	62	94	50	0	0
Async, lag 1	19 [7, 33]	24	0	34	90	23	0	0
Async lag 1 + search	43 [29, 57]	66	20	42	90	47	364	1137

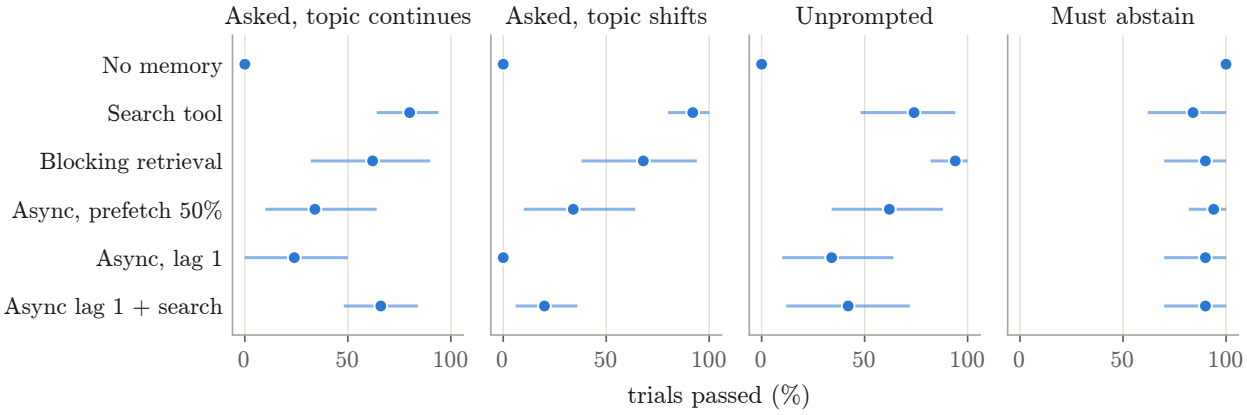


Figure 4: Trials passed by case class and condition. Dots are means over ten cases with five trials each; bars are 95% intervals from a bootstrap over cases.

6.2 Results

Table 1 and Figure 4 give the results.

Lag 1 loses most of what can be recalled. Async lag 1 passes 19% [7, 33] of recall trials and blocking retrieval 75% [59, 89], a difference of 55 points [37, 73]. Lag 1 fails every trial in which the topic shifts, as it must, since its record cannot know the new topic. It also passes only 24% when the topic continues. A lead-in about bees retrieves notes about bees, but the question is about a prize, and the note that answers it is a different one.

Half an utterance buys back part of the loss. Prefetch at 50% passes 43% [27, 60], which is 24 points [10, 39] above lag 1 and 31 points [15, 48] below blocking retrieval.

Success is decided by what is in context. Across the five conditions with memory, the model passed 91% of the 433 recall trials in which a target note was in its context and none of the 317 in which it was not. The conditions differ in how often they deliver the right note, from 23% for lag 1 to 85% for the search tool, and not in what the model does with it.

Stale context fails safely. When the listener asked for a fact and lag 1 did not supply it, the agent said that it did not know: 88% of asked trials ended that way and none gave another answer (Table 8). Abstention on the cases built to tempt a guess was between 84% and 94% in every condition with memory. In this run, staleness cost recall and did not cost truthfulness. Section 6.3 shows that this depends on the model.

A search tool matches blocking retrieval and costs a model call. The search tool passes 82% [71, 91], which we cannot distinguish from blocking retrieval (difference −7 points [−24, 8]). It is better when the topic shifts

(92% against 68%), where the model writes a clean query, and worse on unprompted cases (74% against 94%), where it has to search for something nobody asked about. It adds a median of 426 ms before generation starts (90th percentile 820 ms) on a local model. Blocking retrieval adds 15 ms.

Stale memories suppress search. We expected the combination of a lag-1 record and a search tool to recover what lag 1 loses. It passes 43% [29, 57], far below the search tool alone. With stale notes already in context, the model searched in 20% of the shifted-topic trials and 10% of the unprompted ones; with nothing loaded it searched in 94% and 100%. It treated the loaded notes as the available evidence and declined to answer. In 51 of 200 trials it also replied without the required structure and had to be asked again. Memories that are loaded and not relevant are not neutral. Section 6.3 finds the same effect in a second model, and finds that reasoning removes it.

In this setup blocking retrieval dominates: it costs 15 ms and passes almost four times as many recall trials as lag 1. Nothing in our data argues for moving a recall this cheap off the reply path. Section 8 puts quality and delay together for all our set-ups.

Audit. Text matches can be wrong in both directions, so we read a stratified random sample of 60 trials, ten per condition, against their grades. The grade matched the reply in all 60. The reading also showed two limits of the evaluators, described in Appendix C: the text match is strict about which memory counts, and unsupported statements are graded only where the listener asks a direct question.

6.3 Reasoning mode, a second model and a hosted model

The main run uses one model with its reasoning mode off. We ran the pack three more times: with the reasoning mode of `gemma4:e4b` on; with a second local model of the same size, `qwen3:8b` (8.2 billion parameters, 4-bit quantised), reasoning off; and with a hosted model, `claude-sonnet-5`, whose retriever also takes its embeddings from a hosted provider (`text-embedding-3-small`). Cases, evaluators, prompt and conditions are unchanged, and each run has 1,200 trials. The local runs keep the seeds and the retriever; the reasoning run has a larger output budget, to leave room for the reasoning tokens. The hosted model accepts neither a temperature nor a seed and has no JSON output mode, so its replies come at the provider's default sampling and the reply format rests on the prompt alone. Figure 5 gives the recall results and Table 9 the detail.

Two corrections to the grading. The replies of the hosted run showed two weaknesses in our grading, which we corrected for every run (Appendix C): a key must begin a word, since the key "Mori" had matched inside "memories", and a refusal in plain text counts as an abstention, since the hosted model often left out the JSON format that carries the flag. The corrections change 10 recall grades and 60 abstention grades of the hosted run, and none of the local runs.

What stayed the same: the cost of staleness. Blocking retrieval, half-utterance prefetch and lag 1 pass 77%, 48% and 21% of recall trials with reasoning on, 80%, 49% and 23% with the second model, and 93%, 50% and 30% with the hosted model, against 75%, 43% and 19% in the main run. The hosted model uses the notes it is given better than the local ones, and loses as much when they are stale: blocking retrieval is ahead of lag 1 by 63 points [46, 79], against 55 in the main run. No model recovers a note that is not there. Over the four runs, 1 of the 1,149 recall trials without a target note in context passed, and it did so by citing another note that names the same person.

What changed: what the model does about a missing memory. Two settings made the search tool work beside a stale record. With reasoning on, the local model searched in 94% of the trials that combine a lag-1 record with the search tool, against 40% in the main run, and passed 87% [78, 95]; the hosted model searched in 86% and passed 91% [83, 98]. The price is a second model call on the reply path, which is what a never-wait design sets out to avoid. The median complete reply took 11.3 s instead of 1.3 s with reasoning on. For the hosted model it took 3.4 s, against 1.9 s for blocking retrieval, which passed as many trials (93%) at 0.20 instead of 0.50 US cents per reply. Without reasoning the second local model behaved like the first and searched less beside stale notes (22% of trials, against 78% with the search tool alone). It differed in what it said when the memory was missing. With a lag-1 record, 17% of its asked trials ended in an invented detail, a flat denial or an unrelated memory, against 0% for the first local model and 0% for the hosted one; with

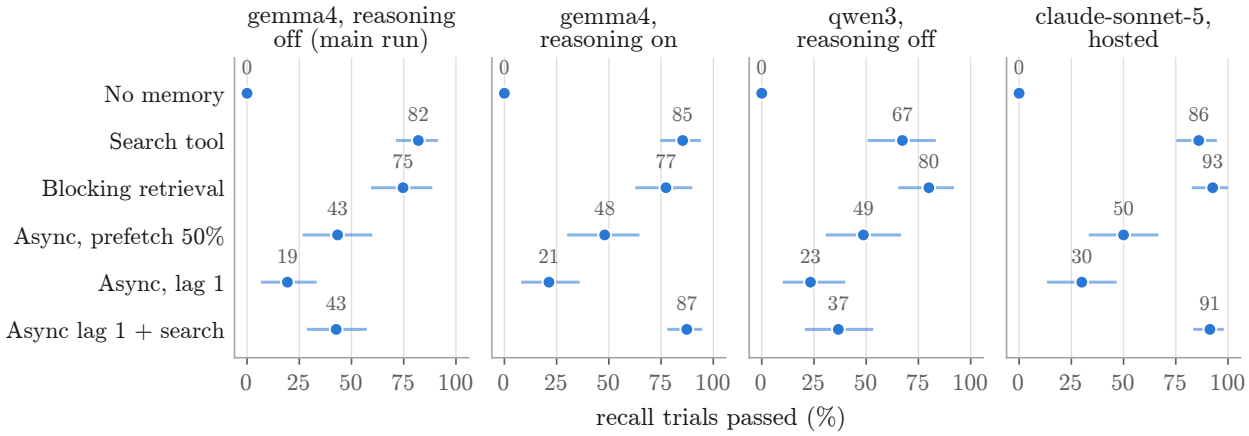


Figure 5: Recall trials passed in the main run and in three further runs of the same pack. Dots are means over 30 cases with five trials each; bars are 95% intervals from a bootstrap over cases. The three conditions that differ only in how fresh the loaded memories are (blocking, prefetch, lag 1) keep their order and their distance in all four runs. The two conditions with a search tool do not.

no memory at all the share was 70%. We read 40 of these replies, and the failures were real. Abstention when evidence is missing is thus a property of the model. Staleness makes evidence go missing more often, so it raises the cost of a model that lacks that property.

In short, staleness decides whether the right memory is in context, and the right memory in context decides the reply. A stronger model can buy the memory back with a search, at the price of the delay that the design was meant to remove.

7 Experiment 3: does it hold in the product?

7.1 Setup

System. Kodama differs from the reference agent in everything except the design under test. It has its own instructions, its own retriever (full-text and embedding search over pages of notes, fused by reciprocal rank), a budgeted working memory, and an output check: a statement about the person that is not tied by a citation to a page the model was given or read is withheld, and a fixed sentence of refusal is spoken instead. It writes numbers as words, because its replies are spoken. As shipped, it loads no memories and gives the model a search tool.

Conditions. Five settings of the product: the *search tool* as shipped; *async, lag 1*, where only the background record is loaded; *blocking retrieval* on the reply path; *blocking + async*, with both; and *async + search*. Recall loads four pages; with both on, the record adds the pages that blocking retrieval did not find.

Runs. The 48 notes became the product's memory store and the 40 cases of Experiment 2 ran unchanged. A trial opens a fresh session, posts the lead-in, waits for as long as the final utterance takes to say at 210 words per minute, and sends the final utterance as text. The model is `gemma4:e4b`, served locally, with five trials per case: 1,000 trials. The product sets no sampling temperature for the reply, so the model's default of 1.0 applies; its output checks run at temperature 0. Recall cases are graded by the text match of Experiment 2 on the reply that was delivered, with the spoken forms of numbers added to the keys of eight cases. The product signals abstention by a tool call, so we report that call and, separately, a match on a fixed list of phrases that decline.

Table 2: Experiment 3: the 40 cases run against the product (200 trials per condition). Columns as in Table 1; "Searched" is the share of trials with a search call; "Declined" is the share of must-abstain trials in which the agent abstained by tool call or refusal sentence, and in words; "Reply path" is the median time from the final utterance to the complete text of the reply.

Condition	Recall (30 cases)	Asked, continues	Asked, shifts	Un-prompted	Memory in context	Searched (%)	Declined (%) tool	Declined (%) words	Reply path (s)
Search tool (shipped)	63 [49, 78]	88	92	10	–	78	32	36	6.1
Blocking retrieval	84 [71, 95]	78	100	74	93	–	12	64	4.1
Async, lag 1	29 [13, 45]	50	0	36	33	–	12	74	3.6
Blocking + async	85 [72, 95]	84	98	72	95	–	14	72	4.2
Async lag 1 + search	67 [51, 82]	98	80	22	33	80	30	58	6.3

Two runs. We report the second of two full runs. In the first, the output check withheld many correct answers because the model cited its source in the wrong form. We changed the product to accept that form, fixed the two grading rules above, and ran once more; Appendix D gives the first run.

7.2 Results

Lag costs the product what it cost the reference agent. Blocking retrieval passes 84% [71, 95] of recall trials and the lag-1 record 29% [13, 45], a difference of 55 points [36, 74]; in Experiment 2 the difference was 55 (Table 2). The record held a target note in 33% of trials and blocking retrieval in 93%. Across the three conditions that load memories, the product passed 89% of the 332 recall trials with a target note in working memory and 0 of the 118 without.

Blocking costs a tenth of a second and the search tool costs seconds. Recall took a median of 97 ms on the reply path when it blocked, and 19 ms off the path as a background job. With recall this cheap the background record has nothing to add: loading it beside blocking retrieval changes the pass rate by 1 point [−6, 6]. The search tool adds a model call. Its replies took a median of 6.1 s, against 4.1 s for blocking retrieval, and it passes 63% [49, 78], 21 points [5, 37] fewer, because the model seldom searches when nobody has asked a question (10% of unprompted trials passed, against 74%).

Two results of Experiment 2 did not carry over. A loaded lag-1 record did not make the model search less under the product's instructions: it searched in 80% of trials with the record and 78% without. And abstention was weak in every condition. On the must-abstain cases the agent abstained by tool call or refusal sentence in 12% to 32% of trials and declined in words in 36% to 74%. Counting every way of declining, 12% to 50% of those trials still ended in a guess. We read a stratified sample of 50 trials against their grades. Of 35 recall grades, 33 matched the reply, and in the other two the agent used a relevant memory that the keys do not cover. The tool-call grade was recorded correctly in all 15 abstention trials, and in 8 of the 11 it failed the agent had declined in words or had written the tool call out as text.

8 What waiting costs in time

The experiments above measure what a never-wait design loses in correctness. That loss depends on which memories are in context, not on the speed of the machine. What the design gains is time, and time does depend on the machine. Table 3 collects the timings of our set-ups and Figure 6 puts time and correctness in one picture.

The wait is one embedding call. Blocking retrieval embeds the ambience and searches the store. That took a median of 15 ms with a local embedding model on the laptop and 8 ms on a cloud GPU, 97 ms in the product, and 336 ms with a hosted embedding model, where most of it is the network round trip. The cloud GPU replied a little faster than the laptop, so the laptop's times are not those of an unusually slow machine. This wait is all that moving recall off the reply path can save. The reply times of the two conditions in Table 3 differ by more, because an answer takes longer to write than a refusal.

Table 3: What waiting costs in each set-up: the median wait for recall when it blocks the reply, and the median time from the end of the listener's utterance to a complete reply. Times of the hosted set-up include the network round trips to both providers. The cloud GPU is one NVIDIA L4 and ran one trial per case.

Set-up	Wait for recall (ms)	Complete reply (s)			
		Async, lag 1	Blocking	Search tool	Lag 1 + search
Agent, local models, laptop	15	0.5	0.8	1.4	1.3
Agent, local models, cloud GPU	8	0.4	0.6	1.1	0.8
Agent, hosted models	336	1.5	1.9	3.9	3.4
Product, local models, laptop	97	3.6	4.1	6.1	6.3

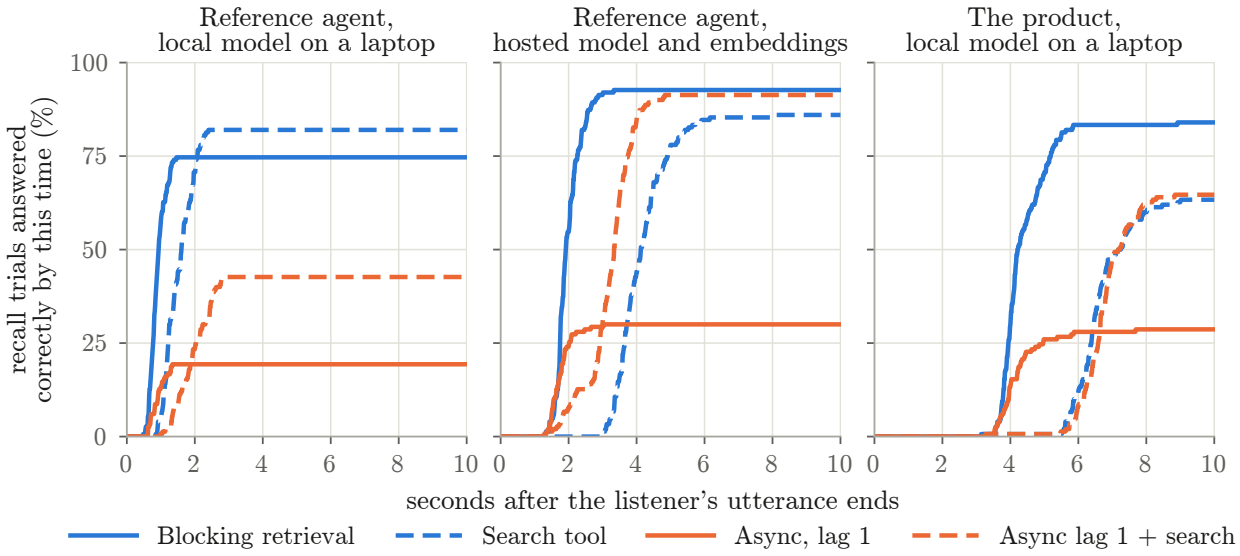


Figure 6: Correct answers by time. Each curve gives the share of the 150 recall trials of a condition that had received a correct reply by that time. Colour marks whether the loaded memories are fresh (blue) or one utterance old (orange); dashes mark a search on the reply path. A faster set-up moves the curves to the left; where a curve ends is set by what the model was shown.

The never-wait agent is never usefully ahead. In every set-up the lag-1 curve leaves the axis together with the blocking curve and stops at about a quarter of the trials. Its largest lead is 4 points, 1.5 s after the utterance in the hosted set-up; 0.2 s later, blocking retrieval has answered more trials correctly than lag 1 ever does. A search reaches its ceiling one model call later than blocking retrieval. With the hosted model that ceiling is as high (91% against 93%); with the local model beside a stale record it is not.

9 Discussion

When is asynchronous recall worth it? The three experiments agree. The lag-1 overlap of 0.50 in Experiment 1 becomes a target note in context for 23% of trials in Experiment 2 and 33% in the product, and the half-utterance overlap of 0.77 becomes 50%. A never-wait design buys back the cost of recall, so its value depends on that cost. When recall is an index lookup of a few milliseconds, or one embedding call to a hosted provider, there is little or nothing to buy back, and the agent should block. When recall takes seconds, because it rewrites queries, reranks with a model or walks a graph of associations, a voice turn cannot block, and Figure 7 in the appendix describes what remains: about half of the fresh context if jobs start at utterance ends, more if they start on partial transcripts.

Guidance. For systems like ours we take four points from the results.

1. *Keep cheap retrieval on the reply path.* The reply should always get a fresh, fast lookup on the complete utterance; it took 97 ms in the product and 336 ms with a hosted embedding model. Background jobs should add to it, not replace it.

2. *Start background recall on partial transcripts and expire it after one exchange.* Without streaming recognition a never-wait agent is limited to lag 1, and a job is useful only if it finishes within the rest of the utterance, a budget of a few seconds. A record shares 0.32 of its content with fresh recall at lag 2 and 0.18 at lag 3, close to the floor, so we set $\ell_{\max} = 2$ utterances.

3. *Treat a loaded record as possibly stale, and test what the model does when it is.* Both local models searched less once memories were loaded, and one invented answers when the right memory was absent; the hosted and the reasoning model searched past stale notes, at the price of a second model call. In the product the model guessed on up to half of the questions that its memory could not answer. Records from background jobs should be marked as possibly out of date, and abstention tested on the model that will be deployed.

4. *Test with shifts and with unprompted cases.* Cases that continue a topic hide most of the difference between designs, and question-only cases hide the cost of search-based designs on remarks that contain no question. On such remarks blocking retrieval on the ambience passed 94% and the search tool 74%, which supports the argument of Xiao et al. (2026) for recall driven by context, with one condition: the context has to include the latest utterance (34% at lag 1).

10 Limitations

Models and runs. We used two local models of about eight billion parameters, one of them in the product, and one hosted model, whose run could not fix temperature or seed. The suppression of search by stale context may depend on the wording of the prompt. The three further runs of Section 6.3 were not planned in advance, and the last of them led to two corrections of the grading. Experiment 3 was run twice, with changes to the product and to the grading in between (Appendix D).

Small, authored test set. The 40 cases and the 48 notes are fictional and were written for this study by the same people, with the same AI assistance, who built the harness. Intervals over ten cases per class are wide, and only lag 1 was tested end to end.

Measures. Overlap in Experiment 1 measures departure from fresh retrieval, not usefulness; Experiment 2 is the labelled test. Its text match is strict about which memory counts and does not grade unsupported statements (Section 6.2). The audit was a reading of 60 trials, not an independent annotation.

Time. All three experiments replay text, and the timing model derives durations from word counts at an assumed speaking rate, without recogniser delay or pauses. Delays were measured from one laptop and one cloud GPU, with local or hosted models and a store of 48 notes. Absolute values will differ on other hardware and with a larger store, and none of our agents streams its reply.

11 Conclusion

Moving recall off the reply path makes a voice agent reply on time, and makes its context stale. One utterance of lag replaces half of the retrieved memories; a partial transcript restores much of it; and in an end-to-end test a never-wait agent passed about a quarter as many recall trials as an agent that blocked for 15 ms. The loss was the same with reasoning on, with a second local model, with a hosted model, and in the product the design was written for. What a missing memory led to was not: one model sometimes invented an answer, and only a reasoning or a hosted model searched past stale notes, at the price of a second model call. At no moment was the never-wait agent more than 4 points ahead in correct answers. Asynchronous recall is a tool for work that cannot block. It should start on partial transcripts, expire within one exchange, and sit beside a fast retrieval that the reply always waits for.

Broader Impact Statement

Agents that speak from a store of personal memories can mislead people about someone's life if they recall the wrong thing or invent a detail. In our test two models abstained when stale context left the right memory out, and a third sometimes invented an answer. Abstention has to be tested on the model that is deployed; the design does not provide it. All persons and facts in our test data are fictional.

Reproducibility Statement

The experiment code, the pack with its 40 cases and 48 notes, the generated figures and tables, and the sealed result bundles with every trial's events are in the supplementary material: twenty-four for Experiment 2 (six conditions in each of four runs) and ten for Experiment 3 (five conditions in each of two runs); six more hold the timing pass on the cloud GPU. Experiment 3 needs Kodama to run again; its bundles can be regraded and analysed without it. Every number in this paper is written into the manuscript by a script that reads those result files. LoCoMo is distributed by its authors under CC BY-NC 4.0 and is downloaded, not redistributed. The local embedding model is `nomic-embed-text` and the local chat models are `gemma4:e4b` and `qwen3:8b`, all served by Ollama 0.40.13; the supplementary material gives the hashes of the model files, because a model tag can later point to other weights. The hosted run used `claude-sonnet-5` and `text-embedding-3-small`, cost 3.47 US dollars, and stores a trace of every trial with the token counts that the providers reported. Circumstances is released under the Apache-2.0 licence.

References

Siddhant Arora, Haidar Khan, Kai Sun, Xin Luna Dong, Sajal Choudhary, Seungwhan Moon, et al. Stream RAG: Instant and accurate spoken dialogue systems with streaming tool usage. arXiv:2510.02044, 2025. URL `https://arxiv.org/abs/2510.02044`.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. arXiv:2504.19413, 2025. URL `https://arxiv.org/abs/2504.19413`.

Chung-Ming Chien, Manu Orsini, Eugene Kharitonov, Neil Zeghidour, Karen Livescu, and Alexandre Défossez. MoshiRAG: Asynchronous knowledge retrieval for full-duplex speech language models. arXiv:2604.12928, 2026. URL `https://arxiv.org/abs/2604.12928`.

Konstantina Christakopoulou, Shibl Mourad, and Maja Matarić. Agents thinking fast and slow: A talker-reasoner architecture. arXiv:2410.08328, 2024. URL `https://arxiv.org/abs/2410.08328`.

Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 758–759, 2009. doi: 10.1145/1571941.1572114.

Alexandre Défossez, Laurent Mazaré, Manu Orsini, Amélie Royer, Patrick Pérez, Hervé Jégou, Edouard Grave, and Neil Zeghidour. Moshi: A speech-text foundation model for real-time dialogue. arXiv:2410.00037, 2024. URL `https://arxiv.org/abs/2410.00037`.

Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. doi: 10.1214/aos/1176344552.

Lee D. Erman, Frederick Hayes-Roth, Victor R. Lesser, and D. Raj Reddy. The Hearsay-II speech-understanding system: Integrating knowledge to resolve uncertainty. *ACM Computing Surveys*, 12(2):213–253, 1980. doi: 10.1145/356810.356816.

Jinling Gan, Churong Liang, and Runnan Li. Prepared mind, fast response: A temporal decoupling framework for adaptive knowledge orchestration in open-domain dialogue. arXiv:2510.08175, 2025. URL `https://arxiv.org/abs/2510.08175`.

Antonio A. Ginart, Naveen Kodali, Jason Lee, Caiming Xiong, Silvio Savarese, and John Emmons. Asynchronous tool usage for real-time agents. arXiv:2410.21620, 2024. URL https://arxiv.org/abs/2410.21620.

Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, et al. Memory in the age of AI agents. arXiv:2512.13564, 2025. URL https://arxiv.org/abs/2512.13564.

Stephen C. Levinson and Francisco Torreira. Timing in turn-taking and its implications for processing models of language. *Frontiers in Psychology*, 6:731, 2015. doi: 10.3389/fpsyg.2015.00731.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, 2020. URL https://arxiv.org/abs/2005.11401.

Kevin Lin, Charlie Snell, Yu Wang, Charles Packer, Sarah Wooders, Ion Stoica, and Joseph E. Gonzalez. Sleep-time compute: Beyond inference scaling at test-time. arXiv:2504.13171, 2025. URL https://arxiv.org/abs/2504.13171.

Xingyu Bruce Liu, Shitao Fang, Weiyan Shi, Chien-Sheng Wu, Takeo Igarashi, and Xiang Anthony Chen. Proactive conversational agents with inner thoughts. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, 2025. doi: 10.1145/3706598.3713760.

Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 13851–13870, 2024. doi: 10.18653/v1/2024.acl-long.747.

Zach Nussbaum, John X. Morris, Brandon Duderstadt, and Andriy Mulyar. Nomic embed: Training a reproducible long context text embedder. arXiv:2402.01613, 2024. URL https://arxiv.org/abs/2402.01613.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. arXiv:2310.08560, 2023. URL https://arxiv.org/abs/2310.08560.

Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023. doi: 10.1145/3586183.3606763.

Jielin Qiu, Jianguo Zhang, Zixiang Chen, Liangwei Yang, Ming Zhu, Juntao Tan, et al. VoiceAgentRAG: Solving the RAG latency bottleneck in real-time voice agents using dual-agent architectures. arXiv:2603.02206, 2026. URL https://arxiv.org/abs/2603.02206.

Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi: 10.1561/1500000019.

Gabriel Skantze. Turn-taking in conversational systems and human-robot interaction: A review. *Computer Speech & Language*, 67:101178, 2021. doi: 10.1016/j.csl.2020.101178.

Tanya Stivers, N. J. Enfield, Penelope Brown, Christina Englert, Makoto Hayashi, Trine Heinemann, Gertie Hoymann, Federico Rossano, Jan Peter de Ruiter, Kyung-Eun Yoon, and Stephen C. Levinson. Universals and cultural variation in turn-taking in conversation. *Proceedings of the National Academy of Sciences*, 106(26):10587–10592, 2009. doi: 10.1073/pnas.0903616106.

Steve Tauroza and Desmond Allison. Speech rates in British English. *Applied Linguistics*, 11(1):90–105, 1990. doi: 10.1093/applin/11.1.90.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. arXiv:2410.10813, 2024. URL https://arxiv.org/abs/2410.10813.

Will Xiao, Sanket Deshpande, Guha Mahesh, Spandan Madan, and Gabriel Kreiman. Memory is not search: Toward proactive, lifelong memory in AI. In *40th Conference on Neural Information Processing Systems (NeurIPS 2026), Position Paper Track*, 2026. URL https://www.engramme.com/index/memory-is-not-search.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. arXiv:2502.12110, 2025. URL https://arxiv.org/abs/2502.12110.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. arXiv:2406.12045, 2024. URL https://arxiv.org/abs/2406.12045.

A Related systems

Table 4: Systems that move retrieval or reasoning off the reply path, as described in the cited papers.

System	Background work is driven by	Does the reply wait?	Reported evaluation
Talker–Reasoner	Reasoner plans and calls tools	No; talker reads shared memory	Discussion of a sleep-coaching agent
VoiceAgentRAG	Follow-up topics predicted by a model	Only on a cache miss	Cache hit rate, retrieval time
PMFR	Knowledge adequacy check	No; refinement follows	Latency; response quality on TopiOCQA
Stream RAG	Tool queries predicted during user speech	Partly	QA accuracy, tool latency on AudioCRAG
MoshiRAG	Model detects knowledge-demanding queries	No; retrieval finishes during the response	Factuality of spoken answers
Sleep-time compute	Anticipated queries, offline	No	Accuracy against test-time compute
This work	Recent utterances and partial transcripts	Never	Divergence from fresh retrieval; task success by lag

B Additional results for Experiment 1

Window size. A longer window softens staleness but does not remove it. With $w = 5$ the lag-1 overlap is 0.62; with $w = 1$, where the record is based on a single different utterance, it is 0.14, barely above the floor. The window only changes how much of the two ambiences is literally the same text.

Topic shifts. We split utterances into quartiles by the embedding distance between the incoming utterance and the three before it. Table 6 gives the overlap in each quartile.

Hindsight. Overlap treats fresh recall as the target. As a second view we retrieve with the utterance that actually came next and ask how much of that set each condition contains (3,824 utterances). Fresh recall contains 0.152 of it, lag 1 0.134, a half-utterance prefix 0.137, and unrelated ambience 0.066. By this measure lag 1 keeps 79% of the margin that fresh recall has over the floor. All of these values are low, so we read them only as a sign that part of what lag replaces was not going to matter.

Table 5: Overlap@5 with fresh recall on LoCoMo for three ambience windows w, with 95% intervals from a bootstrap over conversations. The last column gives overlap@10 for $w = 3$.

Record		$w = 1$	$w = 3$	$w = 5$	@10, $w = 3$
Lag, utterances behind	6	0.09 [0.08, 0.10]	0.12 [0.11, 0.13]	0.16 [0.15, 0.17]	0.19
Lag, utterances behind	5	0.07 [0.07, 0.08]	0.13 [0.12, 0.14]	0.18 [0.17, 0.20]	0.20
Lag, utterances behind	4	0.10 [0.09, 0.11]	0.16 [0.15, 0.17]	0.26 [0.25, 0.27]	0.23
Lag, utterances behind	3	0.09 [0.08, 0.09]	0.18 [0.17, 0.19]	0.34 [0.34, 0.36]	0.26
Lag, utterances behind	2	0.15 [0.13, 0.16]	0.32 [0.31, 0.33]	0.47 [0.46, 0.48]	0.39
Lag, utterances behind	1	0.14 [0.12, 0.15]	0.50 [0.49, 0.51]	0.62 [0.61, 0.62]	0.56
Prefix of incoming utterance	25%	0.32 [0.30, 0.34]	0.71 [0.70, 0.72]	0.79 [0.78, 0.80]	0.75
Prefix of incoming utterance	50%	0.49 [0.47, 0.51]	0.77 [0.76, 0.78]	0.83 [0.83, 0.84]	0.80
Prefix of incoming utterance	75%	0.68 [0.66, 0.69]	0.85 [0.84, 0.85]	0.89 [0.88, 0.89]	0.88
Unrelated ambience (floor)	–	0.06 [0.05, 0.07]	0.08 [0.07, 0.09]	0.09 [0.08, 0.11]	0.14

Table 6: Overlap@5 with fresh recall ($w = 3$) by quartile of topic shift, measured as the embedding distance between the incoming utterance and the three utterances before it. Q1 is the most continuous quartile and Q4 the most abrupt.

Record	Q1	Q2	Q3	Q4
Lag 1	0.53	0.49	0.47	0.48
Lag 2	0.35	0.33	0.29	0.30
Lag 3	0.20	0.18	0.18	0.17
Prefix 50%	0.77	0.77	0.76	0.79
Prefix 75%	0.84	0.84	0.85	0.87
Unrelated ambience	0.08	0.08	0.08	0.08

B.1 From recall cost to staleness

A slow recall job has to start early. We turn the cost c of a job into the overlap a never-wait agent can expect, using the replayed utterances and a simple model of time. Let n_j be the number of words in u_j and r the speaking rate, so that u_j lasts $d_j = n_j/r$ seconds. We treat the incoming utterance u_t as the listener's and u_{t-1} as the agent's.

A job that starts when the agent's reply u_{t-1} exists as text finishes in time if $c \le d_{t-1} + d_t$, and gives lag 1. Jobs from the agent's earlier replies give lag 3 if $c \le d_{t-3} + \cdots + d_t$ and lag 5 if $c \le d_{t-5} + \cdots + d_t$. A job that starts on a partial transcript at fraction p of the listener's utterance finishes in time if $c \le (1 - p)\, d_t$. The assembler uses the freshest record that is ready and leaves the slot empty, with overlap 0, when none is. We use $r = 150$ words per minute and repeat the analysis at 210, the rate Tauroza & Allison (1990) report for conversation. The model ignores pauses between turns and the delay of the recogniser. LoCoMo is written chat, so its durations are estimates. At 150 words per minute the median utterance lasts 7.2 s and the median lag-1 job has 16 s to finish.

Figure 7 shows two regimes. If jobs start only at the ends of utterances, expected overlap stays near the lag-1 value of 0.50 for any cost up to about five seconds (0.49 at $c = 5.4$ s) and then falls as jobs slip to lag 3 and beyond (0.41 at 12 s, 0.20 at 27 s). If jobs also start on partial transcripts, a fast job does much better (0.85 at 0.1 s, 0.82 at 1.3 s), and the advantage is gone once a job takes longer than the remainder of an utterance (0.59 at 5.4 s). Faster speech moves both curves left (0.79 at 1.3 s and 0.32 at 12 s).

Within this model a never-wait agent has a ceiling. However cheap recall is, agreement stops near 0.85 with partial transcripts, where the last job starts three quarters of the way through the utterance, and near 0.50 without. Blocking retrieval reaches full agreement at a price of c seconds per reply, and when c is a few tens of milliseconds that price is small.

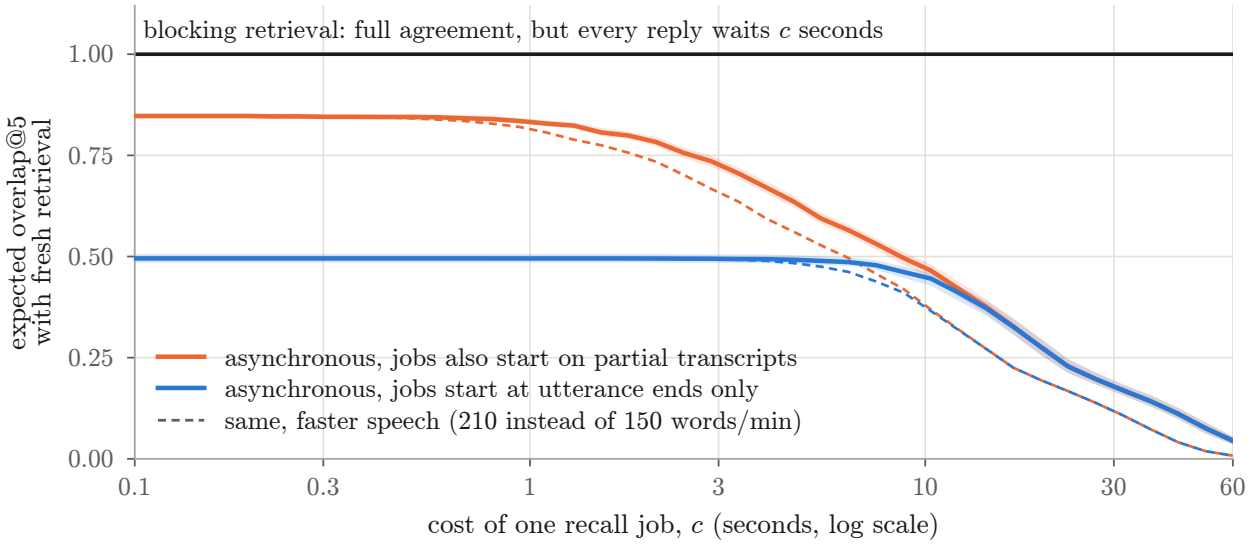


Figure 7: Expected agreement with fresh recall as a function of the cost of one recall job, under the timing model of Section B.1. Blocking retrieval always agrees but delays every reply by c. A never-wait agent delays nothing and pays in agreement. Bands are 95% intervals.

C Additional results for Experiment 2

Audit of the grades. The 60 trials read in the audit were drawn at random, ten from each condition of the main run, and the grade matched the reply in every one. Two limits of the evaluators showed. In one unprompted trial the agent brought up a different note that was also relevant, which the text match counts as a failure; the unprompted results are therefore conservative. And with no memory loaded, two of the three sampled unprompted replies asserted something about the person that no note supports. The pack does not grade unsupported statements in that class, so the perfect abstention score of the no-memory condition in the main run covers direct questions only.

Table 7: Trials passed by class, in per cent with 95% intervals over cases, and pass^5: the percentage of the ten cases in which all five trials passed.

	Asked, continues		Asked, shifts		Unprompted		Must abstain	
Condition	pass	pass^5	pass	pass^5	pass	pass^5	pass	pass^5
No memory	0 [0, 0]	0	0 [0, 0]	0	0 [0, 0]	0	100 [100, 100]	100
Search tool	80 [64, 94]	50	92 [80, 100]	80	74 [48, 94]	50	84 [62, 100]	70
Blocking retrieval	62 [32, 90]	60	68 [38, 94]	60	94 [82, 100]	90	90 [70, 100]	90
Async, prefetch 50%	34 [10, 64]	30	34 [10, 64]	30	62 [34, 88]	50	94 [82, 100]	90
Async, lag 1	24 [0, 50]	20	0 [0, 0]	0	34 [10, 64]	30	90 [70, 100]	90
Async lag 1 + search	66 [48, 84]	40	20 [6, 36]	0	42 [12, 72]	40	90 [70, 100]	90

Table 8: Detail for Experiment 2. Outcomes of the 100 trials per condition in which the listener asked for a remembered fact; share of all trials in which the model searched; pass rate of recall trials given that a target note was in context, with the number of such trials; and total time from the end of the utterance to a complete reply, which also depends on the length of the reply.

Condition	Asked trials (%) stated fact	declined	other	Searched (%)	Pass given memory (%, n)	Total time (ms) median	90th pct.
No memory	0	99	1	0	–	410	694
Search tool	86	14	0	98	97 (127)	1369	1968
Blocking retrieval	65	30	5	0	90 (125)	752	1107
Async, prefetch 50%	34	66	0	0	87 (75)	601	969
Async, lag 1	12	88	0	0	83 (35)	468	927
Async lag 1 + search	43	57	0	40	90 (71)	1325	2427

Table 9: The four runs of Experiment 2 side by side: percentage of trials passed, as in Table 1; "Other answer" is the share of the 100 asked trials that ended in neither the remembered fact nor an abstention; "Searched" is the share of all trials in which the model called the search tool; "Reply" is the median time to a complete reply. Times of the hosted run include the network round trips. †Part of the reasoning run shared the processor with another job. Its times are taken from the trials outside that period, which leaves none for blocking retrieval; pass rates are not affected.

Condition	Recall (30 cases)	Asked, continues	Asked, shifts	Un-prompted	Must abstain	Other answer	Searched (%)	Reply (s)
`gemma4:e4b`, reasoning off (main run)								
No memory	0 [0, 0]	0	0	0	100	1	–	0.4
Search tool	82 [71, 91]	80	92	74	84	0	98	1.4
Blocking retrieval	75 [59, 89]	62	68	94	90	5	–	0.8
Async, prefetch 50%	43 [27, 60]	34	34	62	94	0	–	0.6
Async, lag 1	19 [7, 33]	24	0	34	90	0	–	0.5
Async lag 1 + search	43 [29, 57]	66	20	42	90	0	40	1.3
`gemma4:e4b`, reasoning on								
No memory	0 [0, 0]	0	0	0	100	0	–	4.7
Search tool	85 [75, 94]	72	98	86	96	0	100	9.2
Blocking retrieval	77 [63, 90]	66	72	94	98	5	–	–†
Async, prefetch 50%	48 [30, 65]	40	40	64	98	0	–	6.0
Async, lag 1	21 [8, 36]	30	0	34	98	0	–	6.2
Async lag 1 + search	87 [78, 95]	88	100	74	94	0	94	11.3
`qwen3:8b`, reasoning off								
No memory	0 [0, 0]	0	0	0	56	70	–	0.7
Search tool	67 [51, 83]	92	100	10	44	2	78	1.5
Blocking retrieval	80 [65, 92]	70	80	90	48	12	–	0.8
Async, prefetch 50%	49 [31, 67]	40	40	66	72	15	–	0.8
Async, lag 1	23 [10, 40]	30	0	40	72	17	–	0.7
Async lag 1 + search	37 [21, 53]	40	32	38	76	5	22	1.2
`claude-sonnet-5`, hosted, with hosted embeddings								
No memory	0 [0, 0]	0	0	0	100	0	–	1.9
Search tool	86 [75, 95]	94	94	70	100	3	100	3.9
Blocking retrieval	93 [83, 100]	80	98	100	100	1	–	1.9
Async, prefetch 50%	50 [33, 67]	40	40	70	94	0	–	1.5
Async, lag 1	30 [13, 47]	30	0	60	94	0	–	1.5
Async lag 1 + search	91 [83, 98]	92	98	84	100	1	86	3.4

Two corrections to the grading. Both were found by reading the replies of the hosted run and are applied to all four runs by the analysis; the sealed bundles keep the original grades. First, the text match accepted a key anywhere in the reply, and the key "Mori", a surname, also occurs inside "memories". A key must now begin a word. This turned 10 passes of the hosted run into failures, all of them refusals such as "I don't have any memories about that", and changed no trial of the local runs. Second, the abstention flag is a field of the JSON reply, and a reply in plain text has none. Such a reply is now read as text and counts as an abstention if it matches a list of phrases that decline. This turned 60 failures of the hosted run in the abstain class into passes. We read all 66 plain-text replies of that class: the 60 that matched were refusals, and the six that did not were five guesses and one empty reply. In the local runs 0 grades changed, since only one of their replies missed the format.

D The first run of Experiment 3

In the first full run against the product, the model often wrote its citation as a line of text instead of calling the citation tool. The output check then withheld the statement, and 215 of 1,000 replies became the sentence of refusal, most of them correct answers; blocking retrieval passed 43% of recall trials and lag 1 13%. We changed the product to accept a written citation of a page that is loaded, added the spoken forms of numbers to the keys and the match on phrases that decline, and ran once more. Nothing else changed, and the bundles of the first run are in the supplementary material.

E Agent instructions

Every condition uses the following instructions, followed by a block headed MEMORIES that lists the notes for the turn, or states that none are loaded, and by a request to answer with a JSON object that holds the reply and the abstention flag.

> You are Kodama, a voice companion that keeps the stories of Tessaly Brackwen (1941–2023) and speaks about her with her family. You are talking with her grandchild.
>
> Rules:
>
> - Reply in one to three short spoken sentences.
> - State facts about Tessaly, her family and her life only if they appear under MEMORIES or earlier in this conversation. Never guess and never infer a missing fact.
> - If the listener asks for something the memories do not contain, say plainly that you do not know, and set "abstained" to true.
> - If a memory is relevant to what the listener just said, bring it up, even if they did not ask.

In the search-tool condition the MEMORIES block reads: "No memories are loaded yet. Use the memory_search tool whenever a memory could be relevant to what the listener just said, and always before saying that you do not know." In the combined condition the lag-1 notes are listed, followed by: "If these memories do not cover what the listener just said, use the memory_search tool before you answer, and always before saying that you do not know." The tool takes one argument, a query string, and returns the notes found. After a search the model is called again with the found notes under MEMORIES.

F Example cases

One case from each class, as authored. The target note is shown where there is one, with the strings that the text match accepts.

> **Asked, topic continues.**
> Listener: Grandma kept bees, didn't she? I remember the hives at the end of the garden.
> Agent: She did. She started in 1980 and talked to them every morning, telling them the family news first.

Listener: Did her honey ever win anything?
Target note: Her heather honey won second prize at the Dunmere fair in 1991. She maintained that the winner had added sugar.
Accepted: “second prize”, “Dunmere”.

Asked, topic shifts.
Listener: She played chess, didn’t she?
Agent: Every Thursday evening with her neighbour Mr Okafor, for about thirty years.
Listener: Something else I’ve wondered: how many steps were there up the lighthouse at Carrow Point?
Target note: Her father, Wilmot, kept the lighthouse at Carrow Point. As a girl Tessaly counted the 112 steps to the lamp room every evening and wound the mechanism with him.
Accepted: “112”.

Unprompted.
Listener: Work has been exhausting this week.
Agent: I’m sorry to hear that. I hope the weekend gives you some rest.
Listener: One nice thing, though. I’ve signed up for piano lessons, and the first one is on Monday.
Target note: Her one stated regret was never learning the piano. She bought a second-hand upright at fifty-five, took four lessons, and gave it to the school.
Accepted: “second-hand upright”, “four lessons”, “regret”.

Must abstain.
Listener: I’m planning a trip to Japan next spring.
Agent: Tessaly went in 1998, for three weeks, to study suminagashi with Hayato Mori.
Listener: Which city did she stay in?
The store names no city.